```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
data_dir = '/content/drive/MyDrive/skin-cancer-mnist-ham10000/'

# Verify
import os
print(os.listdir(data_dir))
```

```
['hmnist_28_28_L.csv', 'HAM10000_metadata.csv', 'hmnist_8_8_L.csv', 'hmnist_28_28_RGB.csv', 'hmnist_8_8_RGB.csv', 'HAM10000_images_part_1', 'HAM10000_images_part_2']
```

```
# python libraties
import os, cv2,itertools
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tqdm import tqdm
from glob import glob
from PIL import Image

# pytorch libraries
import torch
from torch import optim,nn
from torch.autograd import Variable
from torch.utils.data import DataLoader,Dataset
from torchvision import models,transforms

# sklearn libraries
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# to make the results are reproducible
np.random.seed(10)
torch.manual_seed(10)
torch.cuda.manual_seed(10)
```

Step 1. Data analysis and preprocessing

```
data_dir = '/content/drive/MyDrive/skin-cancer-mnist-ham10000/'
all_image_path = glob(os.path.join(data_dir, '*', '*.jpg'))
imageid_path_dict = {os.path.splitext(os.path.basename(x))[0]: x for x in all_image_path}
lesion_type_dict = {
    'nv': 'Melanocytic nevi',
    'mel': 'dermatofibroma',
    'bkl': 'Benign keratosis-like lesions ',
    'bcc': 'Basal cell carcinoma',
    'akiec': 'Actinic keratoses',
    'vasc': 'Vascular lesions',
    'df': 'Dermatofibroma'
}
```

```
def compute_img_mean_std(image_paths):
    """
        computing the mean and std of three channel on the whole dataset,
        first we should normalize the image from 0-255 to 0-1
    """

    img_h, img_w = 224, 224
    imgs = []
    means, stdevs = [], []

    for i in tqdm(range(len(image_paths))):
        img = cv2.imread(image_paths[i])
        img = cv2.resize(img, (img_h, img_w))
        imgs.append(img)

    imgs = np.stack(imgs, axis=3)
    print(imgs.shape)

    imgs = imgs.astype(np.float32) / 255.

    for i in range(3):
        pixels = imgs[:, :, i, :].ravel()  # resize to one row
        means.append(np.mean(pixels))
        stdevs.append(np.std(pixels))

    means.reverse()  # BGR --> RGB
    stdevs.reverse()

    print("normMean = {}".format(means))
    print("normStd = {}".format(stdevs))
    return means,stdevs
```

```
norm_mean,norm_std = compute_img_mean_std(all_image_path)
```

```
100%|██████████| 10015/10015 [1:02:03<00:00,  2.69it/s]
(224, 224, 3, 10015)
```

```
df_original = pd.read_csv(os.path.join(data_dir, 'HAM10000_metadata.csv'))
df_original['path'] = df_original['image_id'].map(imageid_path_dict.get)
df_original['cell_type'] = df_original['dx'].map(lesion_type_dict.get)
df_original['cell_type_idx'] = pd.Categorical(df_original['cell_type']).codes
df_original.head()
```

	lesion_id	image_id	dx	dx_type	age	sex	localization	path	cell_type	cell_type_idx
0	HAM_0000118	ISIC_0027419	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2
1	HAM_0000118	ISIC_0025030	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2
2	HAM_0002730	ISIC_0026769	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2
3	HAM_0002730	ISIC_0025661	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2
4	HAM_0001466	ISIC_0031633	bkl	histo	75.0	male	ear	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2

Next steps: Generate code with df_original | New interactive sheet

```
# this will tell us how many images are associated with each lesion_id
df_undup = df_original.groupby('lesion_id').count()
# now we filter out lesion_id's that have only one image associated with it
df_undup = df_undup[df_undup['image_id'] == 1]
```

```
df_undup.reset_index(inplace=True)
df_undup.head()
```

	lesion_id	image_id	dx	dx_type	age	sex	localization	path	cell_type	cell_type_idx
0	HAM_0000001	1	1	1	1	1	1	1	1	1
1	HAM_0000003	1	1	1	1	1	1	1	1	1
2	HAM_0000004	1	1	1	1	1	1	1	1	1
3	HAM_0000007	1	1	1	1	1	1	1	1	1
4	HAM_0000008	1	1	1	1	1	1	1	1	1

Next steps: Generate code with df_undup | New interactive sheet

```
# here we identify lesion_id's that have duplicate images and those that have only one image.
def get_duplicates(x):
    unique_list = list(df_undup['lesion_id'])
    if x in unique_list:
        return 'unduplicated'
    else:
        return 'duplicated'

# create a new colum that is a copy of the lesion_id column
df_original['duplicates'] = df_original['lesion_id']
# apply the function to this new column
df_original['duplicates'] = df_original['duplicates'].apply(get_duplicates)
df_original.head()
```

	lesion_id	image_id	dx	dx_type	age	sex	localization	path	cell_type	cell_type_idx	duplicates
0	HAM_0000118	ISIC_0027419	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2	duplicated
1	HAM_0000118	ISIC_0025030	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2	duplicated
2	HAM_0002730	ISIC_0026769	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2	duplicated
3	HAM_0002730	ISIC_0025661	bkl	histo	80.0	male	scalp	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2	duplicated
4	HAM_0001466	ISIC_0031633	bkl	histo	75.0	male	ear	/content/drive/MyDrive/skin-cancer-mnist-ham10...	Benign keratosis-like lesions	2	duplicated

Next steps: Generate code with df_original | New interactive sheet

```
df_original['duplicates'].value_counts()
```

duplicates	count
unduplicated	5514
duplicated	4501

dtype: int64

```
# now we filter out images that don't have duplicates
df_undup = df_original[df_original['duplicates'] == 'unduplicated']
df_undup.shape
```

```
(5514, 11)
```

```
# now we create a val set using df because we are sure that none of these images have augmented duplicates in the train set
y = df_undup['cell_type_idx']
_, df_val = train_test_split(df_undup, test_size=0.2, random_state=101, stratify=y)
df_val.shape
```

```
(1103, 11)
```

```
df_val['cell_type_idx'].value_counts()
```

cell_type_idx	count
4	883
2	88
6	46
1	35
0	30
5	13
3	8

dtype: int64

```
# This set will be df_original excluding all rows that are in the val set
# This function identifies if an image is part of the train or val set.
def get_val_rows(x):
    # create a list of all the lesion_id's in the val set
    val_list = list(df_val['image_id'])
    if str(x) in val_list:
        return 'val'
    else:
        return 'train'

# identify train and val rows
# create a new colum that is a copy of the image_id column
df_original['train_or_val'] = df_original['image_id']
# apply the function to this new column
df_original['train_or_val'] = df_original['train_or_val'].apply(get_val_rows)
# filter out train rows
df_train = df_original[df_original['train_or_val'] == 'train']
print(len(df_train))
print(len(df_val))
```

```
8912
1103
```

```
df_train['cell_type_idx'].value_counts()
```

cell_type_idx	count
4	5822
6	1067
2	1011
1	479
0	297
5	129
3	107

dtype: int64

```
df_val['cell_type'].value_counts()
```

cell_type	count
Melanocytic nevi	883
Benign keratosis-like lesions	88
dermatofibroma	46
Basal cell carcinoma	35
Actinic keratoses	30
Vascular lesions	13
Dermatofibroma	8

dtype: int64

From From the above statistics of each category, we can see that there is a serious class imbalance in the training data. To solve this problem, I think we can start from two aspects, one is equalization sampling, and the other is a loss function that can be used to mitigate category imbalance during training, such as focal loss.

```
# Copy fewer class to balance the number of 7 classes
data_aug_rate = [15,10,5,50,0,40,5]
for i in range(7):
    if data_aug_rate[i]:
        df_train = pd.concat([df_train] + [df_train.loc[df_train['cell_type_idx'] == i,:]]*( data_aug_rate[i]-1), ignore_index=True)
df_train['cell_type'].value_counts()
```

cell_type	count
Melanocytic nevi	5822
Dermatofibroma	5350
dermatofibroma	5335
Vascular lesions	5160
Benign keratosis-like lesions	5055
Basal cell carcinoma	4790
Actinic keratoses	4455

dtype: int64

```
# # We can split the test set again in a validation set and a true test set:
# df_val, df_test = train_test_split(df_val, test_size=0.5)
df_train = df_train.reset_index()
df_val = df_val.reset_index()
# df_test = df_test.reset_index()
```

Step 2. Model building

```
# feature_extract is a boolean that defines if we are finetuning or feature extracting.
# If feature_extract = False, the model is finetuned and all model parameters are updated.
# If feature_extract = True, only the last layer parameters are updated, the others remain fixed.
def set_parameter_requires_grad(model, feature_extracting):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False
```

```
def initialize_model(model_name, num_classes, feature_extract, use_pretrained=True):
    # Initialize these variables which will be set in this if statement. Each of these
    #   variables is model specific.
    model_ft = None
    input_size = 0

    if model_name == "resnet":
        """ Resnet18, resnet34, resnet50, resnet101
        """
        model_ft = models.resnet50(pretrained=use_pretrained)
        set_parameter_requires_grad(model_ft, feature_extract)
        num_ftrs = model_ft.fc.in_features
        model_ft.fc = nn.Linear(num_ftrs, num_classes)
        input_size = 224


    elif model_name == "vgg":
        """ VGG11_bn
        """
        model_ft = models.vgg11_bn(pretrained=use_pretrained)
        set_parameter_requires_grad(model_ft, feature_extract)
        num_ftrs = model_ft.classifier[6].in_features
        model_ft.classifier[6] = nn.Linear(num_ftrs,num_classes)
        input_size = 224


    elif model_name == "densenet":
        """ Densenet121
        """
        model_ft = models.densenet121(pretrained=use_pretrained)
        set_parameter_requires_grad(model_ft, feature_extract)
        num_ftrs = model_ft.classifier.in_features
        model_ft.classifier = nn.Linear(num_ftrs, num_classes)
```

```
        input_size = 224

    elif model_name == "inception":
        """ Inception v3
        Be careful, expects (299,299) sized images and has auxiliary output
        """
        model_ft = models.inception_v3(pretrained=use_pretrained)
        set_parameter_requires_grad(model_ft, feature_extract)
        # Handle the auxilary net
        num_ftrs = model_ft.AuxLogits.fc.in_features
        model_ft.AuxLogits.fc = nn.Linear(num_ftrs, num_classes)
        # Handle the primary net
        num_ftrs = model_ft.fc.in_features
        model_ft.fc = nn.Linear(num_ftrs,num_classes)
        input_size = 299

    else:
        print("Invalid model name, exiting...")
        exit()
    return model_ft, input_size
```

You can change your backbone network, here are 4 different networks, each network also has sevaral versions. Considering the limited training data, we used the ImageNet pre-training model for fine-tuning. This can speed up the convergence of the model and improve the accuracy.

There is one thing you need to pay attention to, the input size of Inception is different from the others (299x299), you need to change the setting of compute_img_mean_std() function

```
# resnet,vgg,densenet,inception
model_name = 'densenet'
num_classes = 7
feature_extract = False
# Initialize the model for this run
model_ft, input_size = initialize_model(model_name, num_classes, feature_extract, use_pretrained=True)
# Define the device:
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
model = model_ft.to(device)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, pl
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to /root/.cache/torch/hub/checkpoints/densenet121-a639ec97.pth
100%|██████████| 30.8M/30.8M [00:00<00:00, 126MB/s]
Using device: cuda
```

```
norm_mean = (0.49139968, 0.48215827, 0.44653124)
norm_std = (0.24703233, 0.24348505, 0.26158768)
# define the transformation of the train images.
train_transform = transforms.Compose([transforms.Resize((input_size,input_size)),transforms.RandomHorizontalFlip(),
                                      transforms.RandomVerticalFlip(),transforms.RandomRotation(20),
                                      transforms.ColorJitter(brightness=0.1, contrast=0.1, hue=0.1),
                                        transforms.ToTensor(), transforms.Normalize(norm_mean, norm_std)])
# define the transformation of the val images.
val_transform = transforms.Compose([transforms.Resize((input_size,input_size)), transforms.ToTensor(),
                                    transforms.Normalize(norm_mean, norm_std)])
```

```
# Define a pytorch dataloader for this dataset
class HAM10000(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, index):
        # Load data and get label
        X = Image.open(self.df['path'][index])
        y = torch.tensor(int(self.df['cell_type_idx'][index]))

        if self.transform:
            X = self.transform(X)

        return X, y
```

```
# Define the training set using the table train_df and using our defined transitions (train_transform)
training_set = HAM10000(df_train, transform=train_transform)
train_loader = DataLoader(training_set, batch_size=32, shuffle=True, num_workers=4)
# Same for the validation set:
validation_set = HAM10000(df_val, transform=train_transform)
val_loader = DataLoader(validation_set, batch_size=32, shuffle=False, num_workers=4)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker processes in total. Our suggested max number of w
  self.check_worker_number_rationality()
```

```
# we use Adam optimizer, use cross entropy loss as our loss function
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss().to(device)
```

Step 3. Model training

```
# this function is used during training process, to calculation the loss and accuracy
class AverageMeter(object):
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count
```

```
total_loss_train, total_acc_train = [],[]
def train(train_loader, model, criterion, optimizer, epoch):
    model.train()
    train_loss = AverageMeter()
```

```
    train_acc = AverageMeter()
    curr_iter = (epoch - 1) * len(train_loader)
    for i, data in enumerate(train_loader):
        images, labels = data
        N = images.size(0)
        # print('image shape:',images.size(0), 'label shape',labels.size(0))
        images = Variable(images).to(device)
        labels = Variable(labels).to(device)

        optimizer.zero_grad()
        outputs = model(images)

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        prediction = outputs.max(1, keepdim=True)[1]
        train_acc.update(prediction.eq(labels.view_as(prediction)).sum().item()/N)
        train_loss.update(loss.item())
        curr_iter += 1
        if (i + 1) % 100 == 0:
            print('[epoch %d], [iter %d / %d], [train loss %.5f], [train acc %.5f]' % (
                epoch, i + 1, len(train_loader), train_loss.avg, train_acc.avg))
            total_loss_train.append(train_loss.avg)
            total_acc_train.append(train_acc.avg)
    return train_loss.avg, train_acc.avg
```

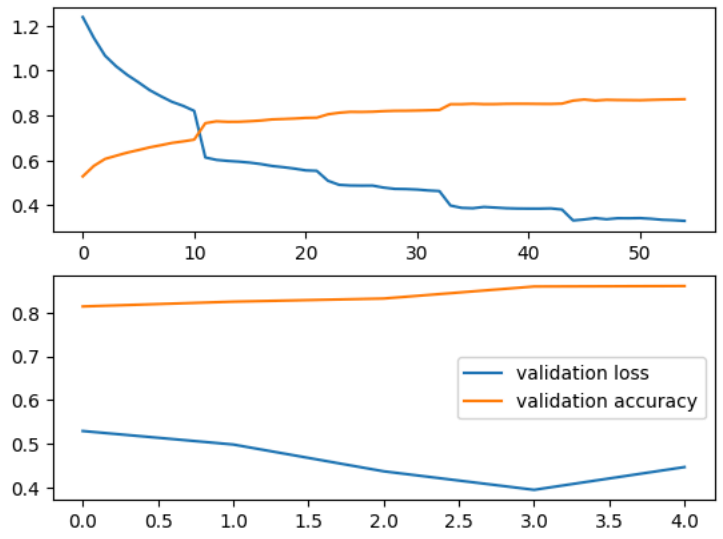
```
def validate(val_loader, model, criterion, optimizer, epoch):
    model.eval()
    val_loss = AverageMeter()
    val_acc = AverageMeter()
    with torch.no_grad():
        for i, data in enumerate(val_loader):
            images, labels = data
            N = images.size(0)
            images = Variable(images).to(device)
            labels = Variable(labels).to(device)

            outputs = model(images)
            prediction = outputs.max(1, keepdim=True)[1]

            val_acc.update(prediction.eq(labels.view_as(prediction)).sum().item()/N)

            val_loss.update(criterion(outputs, labels).item())

    print('------------------------------------------------------------')
    print('[epoch %d], [val loss %.5f], [val acc %.5f]' % (epoch, val_loss.avg, val_acc.avg))
    print('------------------------------------------------------------')
    return val_loss.avg, val_acc.avg
```

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
epoch_num = 5
best_val_acc = 0
total_loss_val, total_acc_val = [],[]
for epoch in range(1, epoch_num+1):
    loss_train, acc_train = train(train_loader, model, criterion, optimizer, epoch)
    loss_val, acc_val = validate(val_loader, model, criterion, optimizer, epoch)
    total_loss_val.append(loss_val)
    total_acc_val.append(acc_val)
    if acc_val > best_val_acc:
        best_val_acc = acc_val
        print('*****************************************************')
        print('best record: [epoch %d], [val loss %.5f], [val acc %.5f]' % (epoch, loss_val, acc_val))
        print('*****************************************************')
```

```
[epoch 2], [iter 1000 / 1124], [train loss 0.55614], [train acc 0.78966]
[epoch 2], [iter 1100 / 1124], [train loss 0.55408], [train acc 0.79068]
------------------------------------------------------------
[epoch 2], [val loss 0.49900], [val acc 0.82476]
------------------------------------------------------------
*****************************************************
best record: [epoch 2], [val loss 0.49900], [val acc 0.82476]
*****************************************************
[epoch 3], [iter 100 / 1124], [train loss 0.50994], [train acc 0.80563]
[epoch 3], [iter 200 / 1124], [train loss 0.49185], [train acc 0.81266]
[epoch 3], [iter 300 / 1124], [train loss 0.48889], [train acc 0.81656]
[epoch 3], [iter 400 / 1124], [train loss 0.48829], [train acc 0.81617]
[epoch 3], [iter 500 / 1124], [train loss 0.48847], [train acc 0.81719]
[epoch 3], [iter 600 / 1124], [train loss 0.47949], [train acc 0.81979]
[epoch 3], [iter 700 / 1124], [train loss 0.47383], [train acc 0.82129]
[epoch 3], [iter 800 / 1124], [train loss 0.47299], [train acc 0.82156]
[epoch 3], [iter 900 / 1124], [train loss 0.47079], [train acc 0.82243]
[epoch 3], [iter 1000 / 1124], [train loss 0.46675], [train acc 0.82369]
[epoch 3], [iter 1100 / 1124], [train loss 0.46409], [train acc 0.82500]
------------------------------------------------------------
[epoch 3], [val loss 0.43790], [val acc 0.83190]
------------------------------------------------------------
*****************************************************
best record: [epoch 3], [val loss 0.43790], [val acc 0.83190]
*****************************************************
[epoch 4], [iter 100 / 1124], [train loss 0.39957], [train acc 0.85062]
[epoch 4], [iter 200 / 1124], [train loss 0.38927], [train acc 0.85062]
[epoch 4], [iter 300 / 1124], [train loss 0.38762], [train acc 0.85260]
[epoch 4], [iter 400 / 1124], [train loss 0.39340], [train acc 0.85086]
[epoch 4], [iter 500 / 1124], [train loss 0.39062], [train acc 0.85106]
[epoch 4], [iter 600 / 1124], [train loss 0.38770], [train acc 0.85229]
[epoch 4], [iter 700 / 1124], [train loss 0.38637], [train acc 0.85268]
[epoch 4], [iter 800 / 1124], [train loss 0.38596], [train acc 0.85250]
[epoch 4], [iter 900 / 1124], [train loss 0.38590], [train acc 0.85201]
[epoch 4], [iter 1000 / 1124], [train loss 0.38690], [train acc 0.85178]
[epoch 4], [iter 1100 / 1124], [train loss 0.38195], [train acc 0.85327]
------------------------------------------------------------
[epoch 4], [val loss 0.39565], [val acc 0.85935]
------------------------------------------------------------
*****************************************************
best record: [epoch 4], [val loss 0.39565], [val acc 0.85935]
*****************************************************
[epoch 5], [iter 100 / 1124], [train loss 0.33319], [train acc 0.86656]
[epoch 5], [iter 200 / 1124], [train loss 0.33763], [train acc 0.87156]
[epoch 5], [iter 300 / 1124], [train loss 0.34371], [train acc 0.86698]
[epoch 5], [iter 400 / 1124], [train loss 0.33864], [train acc 0.87016]
[epoch 5], [iter 500 / 1124], [train loss 0.34316], [train acc 0.86925]
[epoch 5], [iter 600 / 1124], [train loss 0.34290], [train acc 0.86880]
[epoch 5], [iter 700 / 1124], [train loss 0.34369], [train acc 0.86835]
[epoch 5], [iter 800 / 1124], [train loss 0.34072], [train acc 0.86988]
[epoch 5], [iter 900 / 1124], [train loss 0.33622], [train acc 0.87118]
[epoch 5], [iter 1000 / 1124], [train loss 0.33434], [train acc 0.87184]
[epoch 5], [iter 1100 / 1124], [train loss 0.33145], [train acc 0.87284]
------------------------------------------------------------
[epoch 5], [val loss 0.44751], [val acc 0.86036]
------------------------------------------------------------
*****************************************************
```

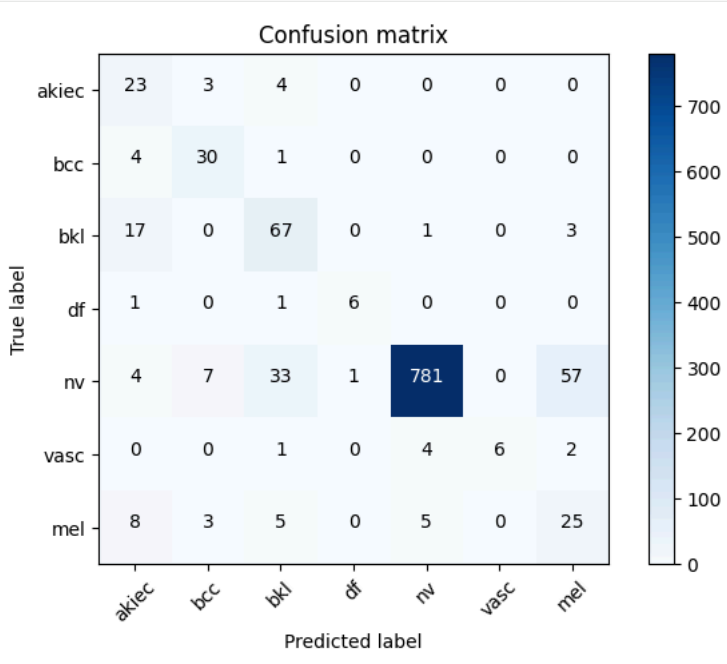
Step 4. Model evaluation

```
fig = plt.figure(num = 2)
fig1 = fig.add_subplot(2,1,1)
fig2 = fig.add_subplot(2,1,2)
fig1.plot(total_loss_train, label = 'training loss')
fig1.plot(total_acc_train, label = 'training accuracy')
fig2.plot(total_loss_val, label = 'validation loss')
fig2.plot(total_acc_val, label = 'validation accuracy')
plt.legend()
plt.show()
```




```
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
```

```
model.eval()
y_label = []
y_predict = []
with torch.no_grad():
    for i, data in enumerate(val_loader):
        images, labels = data
        N = images.size(0)
        images = Variable(images).to(device)
        outputs = model(images)
        prediction = outputs.max(1, keepdim=True)[1]
        y_label.extend(labels.cpu().numpy())
        y_predict.extend(np.squeeze(prediction.cpu().numpy().T))

# compute the confusion matrix
confusion_mtx = confusion_matrix(y_label, y_predict)
# plot the confusion matrix
plot_labels = ['akiec', 'bcc', 'bkl', 'df', 'nv', 'vasc','mel']
plot_confusion_matrix(confusion_mtx, plot_labels)
```



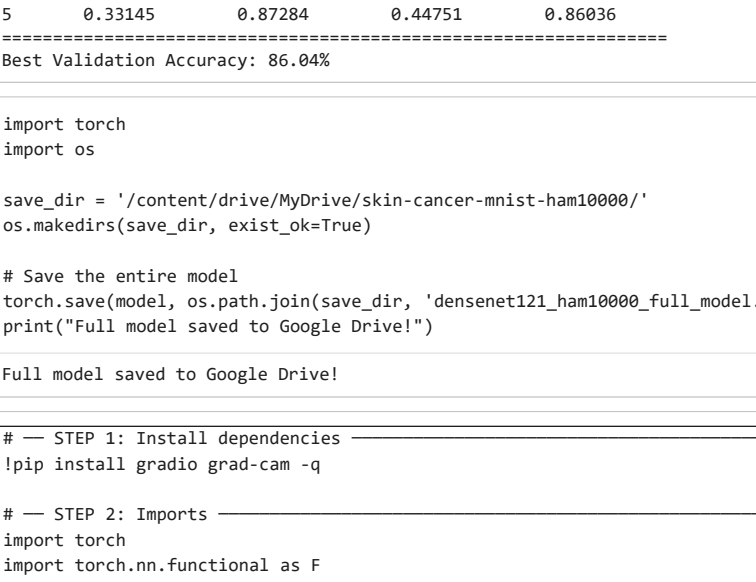

```
# Generate a classification report
report = classification_report(y_label, y_predict, target_names=plot_labels)
print(report)
```

```
              precision    recall  f1-score   support

       akiec       0.40      0.77      0.53        30
         bcc       0.70      0.86      0.77        35
         bkl       0.60      0.76      0.67        88
          df       0.86      0.75      0.80         8
          nv       0.99      0.88      0.93       883
```

```
           vasc       1.00      0.46      0.63        13
            mel       0.29      0.54      0.38        46

       accuracy                           0.85      1103
      macro avg       0.69      0.72      0.67      1103
   weighted avg       0.90      0.85      0.87      1103
```

```
label_frac_error = 1 - np.diag(confusion_mtx) / np.sum(confusion_mtx, axis=1)
plt.bar(np.arange(7),label_frac_error)
plt.xlabel('True Label')
plt.ylabel('Fraction classified incorrectly')
```

```
Text(0, 0.5, 'Fraction classified incorrectly')
```




```
# Get per-epoch train loss and acc
iters_per_epoch = len(total_loss_train) // epoch_num

print(f"{'Epoch':<8}{'Train Loss':<15}{'Train Acc':<15}{'Val Loss':<15}{'Val Acc':<10}")
print(f"{'='*65}")
for i in range(epoch_num):
    idx = (i + 1) * iters_per_epoch - 1
    print(f"{i+1:<8}{total_loss_train[idx]:<15.5f}{total_acc_train[idx]:<15.5f}{total_loss_val[i]:<15.5f}{total_acc_val[i]:<10.5f}")
print(f"{'='*65}")
print(f"Best Validation Accuracy: {best_val_acc*100:.2f}%")
```

```
Epoch   Train Loss     Train Acc      Val Loss       Val Acc
=================================================================
1       0.82098        0.69295        0.52976        0.81381
2       0.55408        0.79068        0.49900        0.82476
3       0.46409        0.82500        0.43790        0.83190
4       0.38195        0.85327        0.39565        0.85935
5       0.33145        0.87284        0.44751        0.86036
=================================================================
Best Validation Accuracy: 86.04%
```

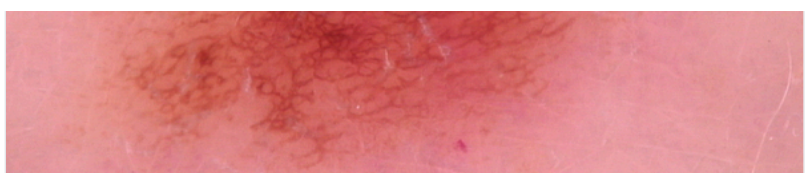
```
import torch
import os

save_dir = '/content/drive/MyDrive/skin-cancer-mnist-ham10000/'
os.makedirs(save_dir, exist_ok=True)

# Save the entire model
torch.save(model, os.path.join(save_dir, 'densenet121_ham10000_full_model.pth'))
print("Full model saved to Google Drive!")
```

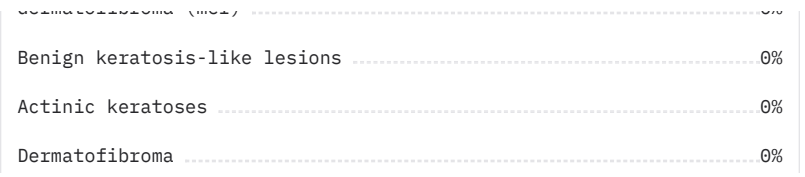
```
Full model saved to Google Drive!
```

```
# ── STEP 1: Install dependencies ──────────────────────────────────
!pip install gradio grad-cam -q

# ── STEP 2: Imports ───────────────────────────────────────────────
import torch
import torch.nn.functional as F
import numpy as np
import cv2
import gradio as gr
from PIL import Image
from torchvision import transforms
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

# ── STEP 3: Config ────────────────────────────────────────────────
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

CLASS_NAMES = {
    0: 'Actinic keratoses',
    1: 'Basal cell carcinoma',
    2: 'Benign keratosis-like lesions',
    3: 'Dermatofibroma',
    4: 'Melanocytic nevi',
    5: 'Vascular lesions',
    6: 'dermatofibroma (mel)'
}

norm_mean = (0.49139968, 0.48215827, 0.44653124)
norm_std  = (0.24703233, 0.24348505, 0.26158768)

# ── STEP 4: Load your saved model ─────────────────────────────────
model = torch.load(
    '/content/drive/MyDrive/skin-cancer-mnist-ham10000/densenet121_ham10000_full_model.pth',
    map_location=device,
    weights_only=False   # ← add this
)
model.eval()

# ── STEP 5: Preprocessing transform ───────────────────────────────
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std)
])

# ── STEP 6: Grad-CAM target layer (last DenseNet conv layer) ──────
target_layer = [model.features.denseblock4.denselayer16.conv2]
cam = GradCAM(model=model, target_layers=target_layer)

# ── STEP 7: Prediction + Grad-CAM function ────────────────────────
def predict_and_gradcam(pil_image):
    # --- Preprocess ---
```

```
    input_tensor = transform(pil_image).unsqueeze(0).to(device)

    # --- Prediction ---
    with torch.no_grad():
        outputs = model(input_tensor)
        probs   = F.softmax(outputs, dim=1).squeeze().cpu().numpy()

    pred_idx   = int(np.argmax(probs))
    pred_label = CLASS_NAMES[pred_idx]
    confidence = float(probs[pred_idx]) * 100

    # --- Confidence scores dict for Gradio bar chart ---
    scores = {CLASS_NAMES[i]: float(probs[i]) for i in range(len(CLASS_NAMES))}

    # --- Grad-CAM heatmap ---
    grayscale_cam = cam(input_tensor=input_tensor)[0]           # (224, 224)

    # Prepare original image as float32 [0,1] for overlay
    img_resized = np.array(pil_image.resize((224, 224))).astype(np.float32) / 255.0
    cam_image   = show_cam_on_image(img_resized, grayscale_cam, use_rgb=True)
    cam_pil     = Image.fromarray(cam_image)

    label_text = f"Prediction: {pred_label}\nConfidence: {confidence:.2f}%"
    return label_text, scores, cam_pil

# ── STEP 8: Gradio UI ──────────────────────────────────────────
with gr.Blocks(title="Skin Cancer Classifier") as demo:
    gr.Markdown("""
    # 🔬 Skin Cancer Classifier — HAM10000
    Upload a dermoscopy image to get a prediction, confidence scores, and a **Grad-CAM heatmap**
    showing which region of the image influenced the model's decision.
    """)

    with gr.Row():
        with gr.Column():
            image_input = gr.Image(type="pil", label="Upload Skin Image")
            run_btn     = gr.Button("Analyze", variant="primary")

        with gr.Column():
            label_out  = gr.Textbox(label="Result", lines=2)
            scores_out = gr.Label(label="Confidence Scores (all classes)")
            cam_out    = gr.Image(label="Grad-CAM Heatmap")

    run_btn.click(
        fn=predict_and_gradcam,
        inputs=image_input,
        outputs=[label_out, scores_out, cam_out]
    )

demo.launch(share=True)   # share=True gives a public link in Colab
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://f8e025b846a3fc2c27.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces
```





Next steps: Deploy to Cloud Run